

高斯泼溅 3DGS 小白指南

从抖音"灵隐寺高斯孪生"作品，到 SIGGRAPH 2023 最佳论文官方开源实现 —— 一文看懂怎么用一堆"会发光的彩色雪球"实时重建真实世界

官方仓库 [graphdeco-inria/gaussian-splatting](#) (24.1k★, 本指南锁定 54c035f)

SIGGRAPH 2023 最佳论文

1080p ≥ 30 FPS 实时渲染

纯 CPU 复刻实验 ☒ 实测跑通

〇 · 起因：一段刷到的视频



抖音上"元游镜相"晒了一条 **高斯孪生应用：灵隐寺** 的视频——这是把整座灵隐寺扫描成了 3DGS 场景，拖动视角就能"云游"实景。作品本身闭源、没有教程，但技术 3DGS 本身有官方开源仓库，值得作为一个学习项目独立开讲。这就是本指南的由来。

同期的民间项目还有"把濒临消失的百座古戏台装进 3D地图"（元游镜相）。**文旅古建数字化**正是 3DGS 落地最猛的方向之一：手机拍一圈照片 → 训练 → 得到可自由视角漫游、照片级真实的 3D 场景。

一 · 疑问即目录：先搞懂"是什么、为什么"

1 3DGS 一句话是什么？解决什么问题？

3DGS (3D Gaussian Splatting, 三维高斯泼溅) 是一种**三维场景表示 + 实时渲染技术**：给一堆从不同角度拍的照片，它重建出整个场景，并且**任何新视角都能实时 (1080p 下 ≥30 FPS) 渲染出照片级画面**。这个过程叫"新视角合成" (Novel View Synthesis)。



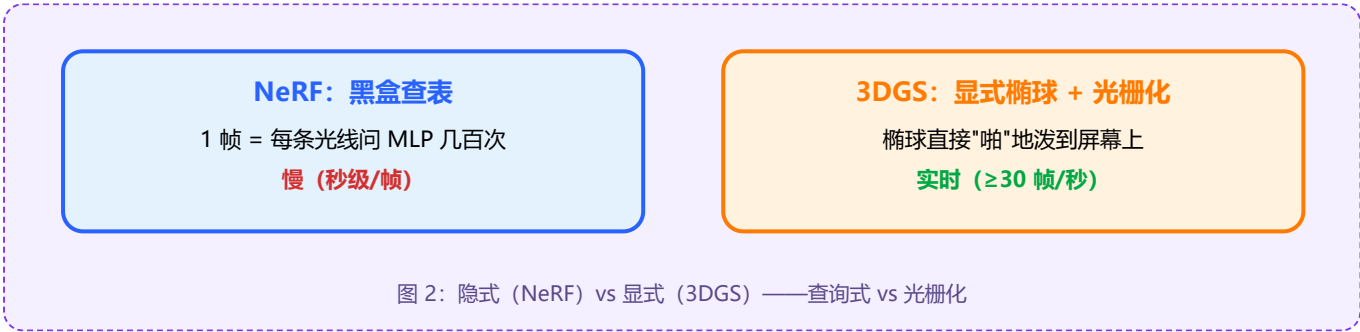
图 1：3DGS 的完整流水线——照片进，可漫游的 3D 场景出

2

它出现之前人们怎么做？为什么大家被它震撼？

路线	怎么做	痛点
传统 mesh 重建 (摄影测量)	照片→稀疏点云→三角面片网格+贴图	反光/半透明/复杂植被重建很烂，质感"塑料"
NeRF (2020) (神经辐射场)	用一个 MLP 神经网络记住整个场景，渲染时每条光线采样几百个点逐一问网络	渲染一帧要数秒~分钟 ，训练数小时~数天
3DGS (2023)	放弃神经网络记场景，改用 上百万个显式的彩色半透明椭球 ，渲染走光栅化老路	实时、训练快、质量还更好 → SIGGRAPH 2023 最佳论文

一句话记住这场革命：NeRF 把场景"藏进神经网络"，查询慢；3DGS 把场景"摊在桌面上"——每个椭球的位置、颜色、大小都看得见摸得着，GPU 光栅化 naturally 快。



二 · 核心原理：一个"高斯"和一场"泼溅"

3

"一个高斯"到底是什么？它在文件里存了什么？

每个高斯就是一个**3D 概率雪球**：中心最"浓"，向外按高斯函数指数衰减到透明。数学上 $G(x)=\exp(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu))$ ——不用怕这个公式，它只说两件事： μ （中心在哪）和 Σ （朝哪扁、朝哪长）。百万个这样的雪球按不同浓度叠在一起，就是照片级场景。

存储字段（.ply 里 62 个 float32）	含义	渲染时的还原
<code>x,y,z</code>	中心位置 μ	直接用
<code>scale_0..2</code>	三根轴的长度（半轴）	存的是 log ，用时 <code>exp</code> 还原（保证永远为正）
<code>rot_0..3</code>	朝向（单位四元数 w,x,y,z ）	归一化后旋转矩阵 R
<code>opacity</code>	不透明度	存的是 logit ，用时过 <code>sigmoid</code> 压到 0~1
<code>f_dc_0..2</code>	基础色（球谐 SH 的直流项）	<code>color = 0.5 + 0.2821×f_dc</code>
<code>f_rest_0..44</code>	球谐高阶项（随视角变的颜色）	次数越高越闪，如高光
<code>nx,ny,nz</code>	法线占位	3DGS 不用，全 0

算笔账（实测）：62 属性 × 4 字节 = **248 字节/个高斯**。实验 1 里我手工构造了 1500 个高斯 = 372KB；真实场景 100 万~500 万个高斯，一个 .ply 就是 **250MB~1.2GB**——"3DGS 文件大"是工程落地的头号问题，也是压缩研究（量化/SOG）的用武之地（类比 [[量化与GGUF]]：LLM 权重有损压缩同族思路）。

4

训练是怎么回事？照片怎么一步步变成高斯云？



图 3：训练四步。注意 ① 的"点云"就是灵隐寺扫描数据的入口

训练的目标函数 = **L1 逐像素差** + **(1-SSIM) 结构差**（权重 $\lambda_{dssim}=0.2$ ），共 3 万次迭代。训练中每隔一段做一次**自适应密度控制**，这是 3DGS 画质好的第二把钥匙，见下一问。

5 场景细节怎么越来越清楚？（克隆 / 分裂 / 剪枝）

训练中统计每个高斯的**位置梯度累积值**——梯度大 = "这个高斯罩不住这一片，照片之间对不齐"。然后分三种手术：

操作	什么情况	动作
克隆 clone (源码 435 行)	梯度大且 体积小 （欠重建，细节没盖住）	原样复制一份挪过去，数量 $\times 2$
分裂 split (源码 409 行)	梯度大且 体积大 （过重建，一个胖球糊了一大片）	缩到 0.8 倍拆成 2 个小的
剪枝 prune (源码 452 行)	透明度 < 0.005 ，或屏幕上/世界里过大	直接删掉

通俗版：像画家修一幅画——哪里画淡了（欠重建）**再点几笔**（克隆），哪里涂成一坨（过重建）**抹开画细**（分裂），**废笔擦掉**（剪枝）。默认在第 500~15000 迭代间每 100 步做一轮。这段逻辑全部在 `scene/gaussian_model.py`，是全仓库最值得精读的 60 行。

6 颜色怎么随视角变？（球面谐波 SH）

同一片水面，正面看是蓝的、侧面反射天光是白的——这叫**视角相关颜色**。3DGS 给每个高斯不发一个固定颜色，而是发一组**球面谐波（Spherical Harmonics）系数**（DC 项 + 45 个高阶项，最高 3 阶）。SH 就是"定义在球面上的傅里叶"：低阶给平均色，高阶给高光闪烁。训练从 0 阶开始逐步"解锁"到 3 阶（源码 `oneupSHdegree`），先学大块再学细节。

连回已有知识：[[向量与Embedding]] 是"用一串数表示一个语义"，SH 系数是"用一串数表示一个球面上的颜色函数"——都是**把连续的东西离散成可优化的参数向量**。实验 1 里 45 个 `f_rest` 全填 0，渲染出来就只剩基础色（无视角特效），一样能拼出完整画面。

7 "泼溅"是怎么画到屏幕上的？（渲染四步）

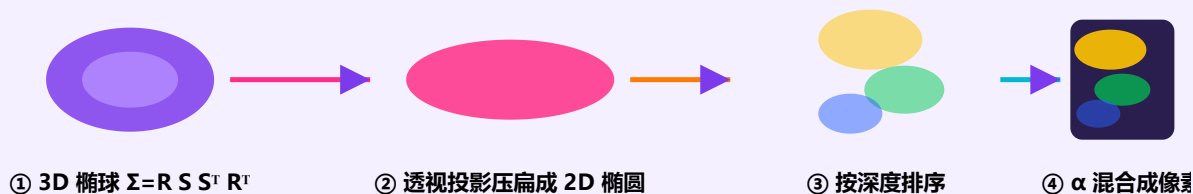


图 4：泼溅渲染四步——本指南实验 2 用纯 numpy 完整复刻的就是这四步

① 由存的 scale/四元数重建 3D 协方差 $\Sigma = R S S^T R^T$ ；② 雅可比矩阵把 3D 协方差"压"成屏幕上的 2D 协方差（论文式 9）；③ **按深度排序**（官方 GPU 版逐 16×16 tile 排，一帧千万级高斯毫秒完成）；④ 逐像素**前向 α 混合**： $C = \Sigma c_i \alpha_i \cdot \Pi(1 - \alpha_j)$ ，近的先画，透过率乘性衰减，天然处理半透明（灵隐寺香火烟雾就是这么"透"出来的）。

3DGS 的高斯云本质是一套“点精灵粒子系统 + 特殊着色”的极端版：

[[粒子系统]] (粒子玫瑰)		3DGS
基本粒子	一个 <code>gl_Point</code> 大点	各向异性椭圆 (可拉长压扁旋转)
数据来源	参数方程算出来	照片训练优化出来
颜色	顶点色/uniform	球面谐波, 随视角变
渲染	GPU 画点	GPU 排序+泼溅光栅化
数量级	3.4 万	百万~千万级

生态上两者已经合流：**Three.js 有 antropic/3dgs 渲染插件 (如 mkkellogg/GaussianSplats3D)**，网页直接播 `.ply/.splat`——你学完 `[[Three.js与场景图]]` 后完全可以在网页里“云游灵隐寺”。

三 · 这个项目给我什么 (能力清单)

① 看懂 3DGS 全家桶生态

官方实现、NVIDIA gsplat、各压缩/加速分支、网页播放器——能按需求选型，不再“见词懵”。

② 评估“我能不能做”的算力账

训练要 CUDA GPU (论文级 24GB 显存, 缩水配置约 8GB)；渲染/查看要求低得多。灵隐寺类文旅项目 = 采集 (手机) + 训练 (云端租卡) 两段。

③ 看懂 .ply 文件、会写解析器

62 字段布局、激活函数 (log/logit) ——拿到任何 3DGS 资产都能读、能批处理、能转格式。

④ 理解“照片→可漫游3D”的完整原理链

SfM 定位→初始化→可微渲染闭环→密度化精修。这套“渲染和真值对比再反传”的思路在自动驾驶仿真、数字孪生、影视虚拟制片通用。

⑤ 判断商业红线

官方代码与光栅化器是 INRIA

非商业研究许可

，商用需授权；生态里有更宽松许可的替代实现。

⑥ 与已有技能树合流

用 Three.js 网页播放器把 3DGS 场景嵌进浏览器——“粒子系统→点精灵→高斯椭圆”一条线打通。

四 · 动手实验 (全部实测跑通, 无 GPU、纯 CPU)

实验设计说明：官方训练需要 CUDA GPU (本机无)，故实验走轻量复刻路线——不训练，而是把“一个 3DGS 资产长什么样、渲染数学是什么”亲手拆开。三个实验环环相扣：**造数据** → **解析数据** → **复刻渲染管线**。环境：Python 3.14 + numpy + Pillow (仅这两个三方库)。

实验 1 · 亲手"造"一个 3DGS 场景: 合成 PLY 点云 (make_synthetic_ply.py)

操作 `cd F:\reminder\gaussian-splatting\exercise` → `python make_synthetic_ply.py`

目的 搞清"一个高斯在文件里存哪些字段"。脚本构造 1500 个高斯: 1000 个红-蓝渐变球壳(模拟实体表面) + 500 个绿色大扁椭圆球(模拟半透明散射片, 像灵隐寺的香火烟雾), 按 3DGS 标准 62 字段写二进制 PLY。

实测结果

OK: synthetic_scene.ply written, 1500 gaussians, 62 fields each, 372000 bytes payload
字段清单: ['x','y','z','nx','ny','nz','f_dc_0','f_dc_1','f_dc_2','f_rest_0'] ... ['rot_3']

验证 ☒ **文件生成、62 字段齐全、372000 字节 = 1500 × 248 字节**——"一个高斯 62 个 float32"从数据结构层坐实。

实验 1b · 零依赖解析器: 把"雪球档案"读回来 (parse_ply.py)

操作 `python parse_ply.py synthetic_scene.ply`

目的 手工解析二进制 PLY (不借助任何 ply 库), 验证对存储格式的理解, 并把 log/logit 编码还原成人能看的参数。

实测结果

[header] vertex 数 = 1500, 每点 62 个 float32 属性 = 248 字节
[payload] 372000 字节 = 1500 × 248 ? True
高斯#0: 位置=(-0.768, 0.557, 0.294) 颜色RGB=(0.81, 0.2, 0.59) 三轴尺度=(0.02,0.02,0.02) 不透明度=0.924
[统计] 包围盒 x[-1.00,1.00] y[-1.15,1.01] z[-1.02,1.01]
OK: 解析通过 — 一个 3DGS 高斯 = 62 个 float32 (62属性 × 4字节 = 248 字节/个)

验证 ☒ **尺度 log→exp、透明度 logit→sigmoid、SH DC→RGB 全部还原正确 (球壳高斯尺度 0.02、盘状 0.15/0.008/0.15 与构造值一致)**

实验 2 (主实验) · 纯 numpy 复刻泼溅渲染管线: mini_splat.py

操作 `python mini_splat.py` → 生成 `render.png`

目的 不用任何 GPU/官方光栅化器, 亲手实现论文渲染四步: $\Sigma = R S S^T R^T \rightarrow$ 雅可比透视投影成 2D 协方差 $\rightarrow$ 深度排序 $\rightarrow$ 前向 α 混合。与官方 `gaussian_model.py:33-47` 的激活函数、`build_covariance_from_scaling_rotation` 逐行对应。

实测结果

载入 1500 个高斯, 开始渲染 512x512 ...
OK: render.png 已生成。1500/1500 个高斯参与了泼溅, 背景透过率均值 0.694

验证 ☒ **打开 render.png: 应看到红蓝渐变的高斯球壳 + 底下一片亮绿色散射盘 (半透明叠色正确), 全图无三角面片——全是椭圆"泼"出来的。透过率 0.694 说明背景占比合理, α 混合没有爆掉。**

实验 3 • 概念→源码定位：官方仓库精读地图

操作 在 `repo\` 里 grep 定位（本指南已实测）：

```
# 密度化主循环    train.py:167-171    add_densification_stats → densify_and_prune
# 三种手术        scene/gaussian_model.py:409 densify_and_split / :435 densify_and_clone / :452 densify_and_prune
# 激活函数        scene/gaussian_model.py:39-47 scaling=torch.exp, opacity=torch.sigmoid
# 协方差构建      scene/gaussian_model.py:33-34 L = build_scaling_rotation(scaling, rotation)
# 渲染入口        gaussian_renderer/__init__.py:91 rasterizer(...) → rendered_image, radii
```

目的 把论文术语钉到具体代码行，建立“论文→代码”翻译表。

验证 ☒ 打开 `densify_and_clone/split`：克隆=原样复制（体积小•欠重建），分裂= $\text{scale}/0.8N$ 拆两个（体积大•过重建），剪枝= $\text{opacity}<0.005$ ——与第 5 问表格完全吻合

下一步升级路线（需要 GPU 时）：租一张 24GB 显显卡 → 按 README `conda env create --file environment.yml`（CUDA 11.8、克隆要 `--recursive` 带 submodules）→ 用手机环拍一个小物件 100~200 张照片 → `python convert.py -s <目录>`（COLMAP）→ `python train.py -s <目录> --eval` → 训完用网页播放器在 Three.js 里展示，就是自己的“灵隐寺时刻”。

信息来源：官方仓库 github.com/graphdeco-inria/gaussian-splatting（SIGGRAPH 2023 最佳论文官方实现，24.1k★，本指南锁定 commit 54c035f） | 论文“3D Gaussian Splatting for Real-Time Radiance Field Rendering”（Kerbl et al., INRIA） | 抖音“元游镜相”灵隐寺高斯孪生作品（学习动机，作品闭源） | smzdm《高斯泼溅卷到手机用户》行业盘点（文旅古建应用背景） | 实验全部于 2026-10-02 在本机 Python 3.14 实测跑通。**License 提示**：官方代码与光栅化器为 INRIA 非商业研究许可，商用需授权。